МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ З

ЛАБОРАТОРНОЇ №1

Варіант 15

на тему **" Основи функціонального програмування у Python "**

Виконала:

Студентка 2 курсу групи ФЕП-22

Сулятицька Христина

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів-2026

Мета. ознайомитися з поняттями чистої функції та референтної прозорості; навчитися виявляти та ізолювати побічні ефекти; реалізувати обробку даних у функціональному стилі; навчитися використовувати typing.Callable для параметризації обчислень; не змінювати вхідні дані та створювати нові об'єкти під час обробки.

Теоретичні відомості.

Чиста функція — це функція, яка для однакових вхідних даних завжди повертає однаковий результат і не змінює зовнішній або глобальний стан програми.

Референтна прозорість означає, що результат виконання виразу можна замінити його значенням без зміни поведінки програми.

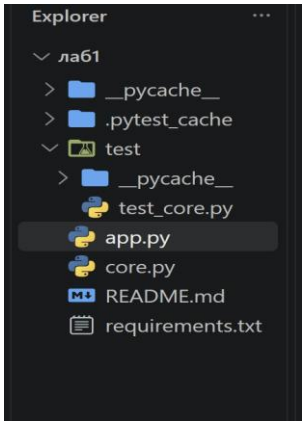
Побічні ефекти — це дії функції, які впливають на зовнішнє середовище: виведення через print, робота з файлами, мережею, зміна глобальних змінних або вхідних структур даних.

У функціональному стилі обчислювальну частину програми доцільно відокремлювати від введення та виведення даних. Вхідні об'єкти не змінюються, а замість них створюються нові об'єкти. Lab_01

Callable використовується для передавання функцій як аргументів. Це дозволяє окремо задавати правила фільтрації, конвертації та комісії й змінювати їх без зміни основного алгоритму обробки. Для 15-го варіанта через Callable передаються функції агрегації результатів спортивних спроб, нарахування бонусу, обчислення штрафу та перевірки кваліфікації спортсмена. Це дозволяє змінювати правила оцінювання без зміни основного алгоритму обробки спортивних результатів.

Хід роботи

1. **Створено структуру проєкту** лабораторної роботи:



2. Реалізовано чисте ядро програми у файлі core.py. У ньому створено функції для:

- обчислення середнього результату спортивних спроб;
- визначення бонусу залежно від категорії спортсмена;
- обчислення штрафу залежно від кількості спроб;
- перевірки кваліфікації спортсмена;
- створення нового об'єкта з обробленими результатами;
- формування налаштованого обробника спортивних результатів.

Реалізовано обробку спортивних результатів. Програма спочатку відбирає лише спортсменів зі статусом completed. Для кожного учасника обчислюється середній результат усіх спроб. Після цього залежно від категорії визначається бонус, а залежно від кількості спроб — можливий штраф.

Обчислено підсумковий результат. Від середнього результату віднімається штраф і додається бонус. Отримане значення використовується для перевірки кваліфікації спортсмена відповідно до його категорії.

Забезпечено відсутність мутації вхідних даних. Замість зміни початкового словника спортсмена створюється новий словник із додатковими полями aggregated_result, bonus, penalty, final_result та qualified.

Використано Callable. Функції агрегації, нарахування бонусу, обчислення штрафу та перевірки кваліфікації передаються в make_processor як аргументи. Це дає можливість змінювати правила оцінювання без зміни основного алгоритму.

```python
from typing import TypedDict, Callable, Iterable


class Athlete(TypedDict):
    id: int
    name: str
    attempts: list[float]
    status: str
    category: str


class AthleteResult(Athlete, total=False):
    average_result: float
    bonus: float
    penalty: float
    final_result: float
    qualified: bool


# Типи функцій для передачі політик
AggregationFn = Callable[[list[float]], float]
BonusFn = Callable[[str], float]
PenaltyFn = Callable[[Athlete], float]
QualificationFn = Callable[[float, str], bool]


# 1. Обчислення середнього результату спроб
def calculate_average(attempts: list[float]) -> float:
    if not attempts:
        return 0.0

    return sum(attempts) / len(attempts)


# 2. Отримання бонусу залежно від категорії
def calculate_bonus(category: str) -> float:
    bonuses = {
        "junior": 5.0,
        "senior": 10.0,
        "professional": 15.0
    }

    return bonuses.get(category, 0.0)


# 3. Обчислення штрафу залежно від кількості спроб
def calculate_penalty(athlete: Athlete) -> float:
    attempts_count = len(athlete["attempts"])

    if attempts_count > 5:
        return 5.0

    if attempts_count == 0:
        return 10.0

    return 0.0


# 4. Перевірка кваліфікації спортсмена
def check_qualification(
    final_result: float,
    category: str
) -> bool:
    minimum_results = {
        "junior": 30.0,
        "senior": 50.0,
        "professional": 70.0
    }

    minimum = minimum_results.get(category, 50.0)

    return final_result >= minimum


# 5. Створення нового запису з результатами
def create_result(
    athlete: Athlete,
    average: float,
    bonus: float,
    penalty: float,
    final_result: float,
    qualified: bool
) -> AthleteResult:

    return {
        **athlete,
        "attempts": list(athlete["attempts"]),
        "average_result": average,
        "bonus": bonus,
        "penalty": penalty,
        "final_result": final_result,
        "qualified": qualified
    }
```

```python


# 6. Функція вищого порядку — фабрика обробника
def make_processor(
    *,
    aggregate: AggregationFn,
    apply_bonus: BonusFn,
    apply_penalty: PenaltyFn,
    qualify: QualificationFn
) -> Callable[[Iterable[Athlete]], dict[str, object]]:

    def process(
        athletes: Iterable[Athlete]
    ) -> dict[str, object]:

        results = []

        for athlete in athletes:

            # Обробляємо тільки завершені виступи
            if athlete["status"] != "completed":
                continue

            average = aggregate(athlete["attempts"])

            bonus = apply_bonus(athlete["category"])

            penalty = apply_penalty(athlete)

            final_result = average + bonus - penalty

            qualified = qualify(
                final_result,
                athlete["category"]
            )

            new_athlete = create_result(
                athlete,
                average,
                bonus,
                penalty,
                final_result,
                qualified
            )

            results.append(new_athlete)

        qualified_count = sum(
            1 for athlete in results if athlete["qualified"]
        )

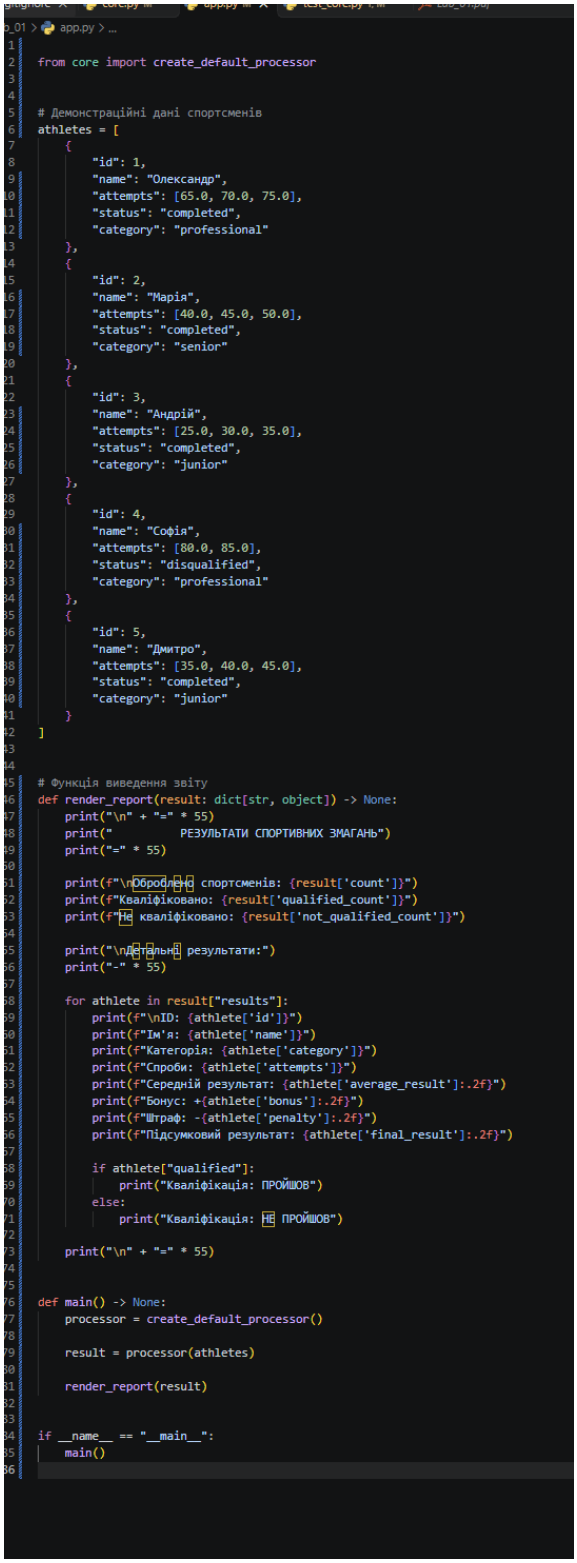
        return {
            "count": len(results),
            "qualified_count": qualified_count,
            "not_qualified_count": len(results) - qualified_count,
            "results": results
        }

    return process


# 7. Створення стандартного обробника
def create_default_processor() -> Callable:
    return make_processor(
        aggregate=calculate_average,
        apply_bonus=calculate_bonus,
        apply_penalty=calculate_penalty,
        qualify=check_qualification
    )
```

3. Реалізація оболонки програми у файлі app.py.

У файлі app.py реалізовано зовнішню оболонку програми, яка відповідає за запуск обробки спортивних результатів та виведення звіту. Для цього використовується функція create_default_processor() з файлу core.py, яка створює налаштований обробник. У програмі задано список спортсменів із такими даними: ідентифікатор, ім'я, результати спроб, статус участі та категорія. Після передавання цих даних до обробника програма отримує результати обчислень. Для відображення інформації створено функцію render_report(), яка виводить кількість оброблених спортсменів, кількість кваліфікованих і некваліфікованих учасників, а також детальні результати кожного спортсмена. Таким чином, введення та виведення даних відокремлено від обчислювального ядра, а побічні ефекти зосереджено у файлі app.py.



```python
from core import create_default_processor


# Демонстраційні дані спортсменів
athletes = [
    {
        "id": 1,
        "name": "Олександр",
        "attempts": [65.0, 70.0, 75.0],
        "status": "completed",
        "category": "professional"
    },
    {
        "id": 2,
        "name": "Марія",
        "attempts": [40.0, 45.0, 50.0],
        "status": "completed",
        "category": "senior"
    },
    {
        "id": 3,
        "name": "Андрій",
        "attempts": [25.0, 30.0, 35.0],
        "status": "completed",
        "category": "junior"
    },
    {
        "id": 4,
        "name": "Софія",
        "attempts": [80.0, 85.0],
        "status": "disqualified",
        "category": "professional"
    },
    {
        "id": 5,
        "name": "Дмитро",
        "attempts": [35.0, 40.0, 45.0],
        "status": "completed",
        "category": "junior"
    }
]


# Функція виведення звіту
def render_report(result: dict[str, object]) -> None:
    print("\n" + "=" * 55)
    print("          РЕЗУЛЬТАТИ СПОРТИВНИХ ЗМАГАНЬ")
    print("=" * 55)

    print(f"\nОброблено спортсменів: {result['count']}")
    print(f"Кваліфіковано: {result['qualified_count']}")
    print(f"Не кваліфіковано: {result['not_qualified_count']}")

    print("\nДетальні результати:")
    print("-" * 55)

    for athlete in result["results"]:
        print(f"\nID: {athlete['id']}")
        print(f"Ім'я: {athlete['name']}")
        print(f"Категорія: {athlete['category']}")
        print(f"Спроби: {athlete['attempts']}")
        print(f"Середній результат: {athlete['average_result']:.2f}")
        print(f"Бонус: +{athlete['bonus']:.2f}")
        print(f"Штраф: -{athlete['penalty']:.2f}")
        print(f"Підсумковий результат: {athlete['final_result']:.2f}")

        if athlete["qualified"]:
            print("Кваліфікація: ПРОЙШОВ")
        else:
            print("Кваліфікація: НЕ ПРОЙШОВ")

    print("\n" + "=" * 55)


def main() -> None:
    processor = create_default_processor()

    result = processor(athletes)

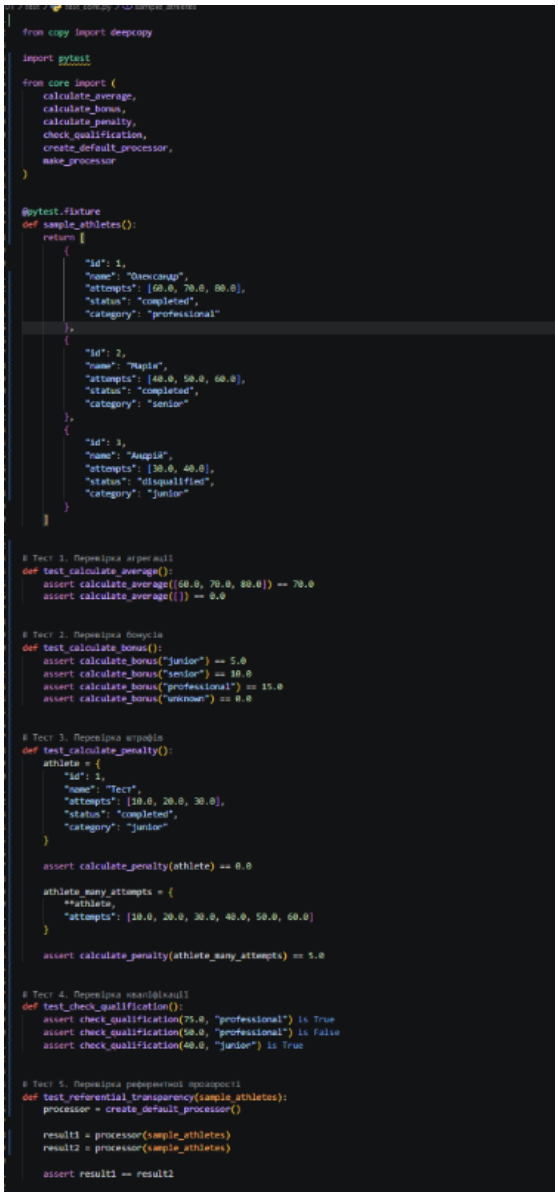
    render_report(result)


if __name__ == "__main__":
    main()
```

4. Створено тести pytest для перевірки:

- референтної прозорості під час обробки спортивних результатів;
- відсутності зміни початкових даних спортсменів;
- правильності обчислення середнього результату, бонусу та штрафу;
- правильності визначення кваліфікації;
- коректної роботи переданих Callable-функцій;
- фільтрації спортсменів за статусом участі.

Виконано тестування програми за допомогою команди `pytest -v`. Перевірено правильність обчислень, відсутність мутації вхідних даних та роботу функцій вищого порядку.



```python
from copy import deepcopy

import pytest

from core import (
    calculate_average,
    calculate_bonus,
    calculate_penalty,
    check_qualification,
    create_default_processor,
    make_processor
)


@pytest.fixture
def sample_athletes():
    return [
        {
            "id": 1,
            "name": "Олександр",
            "attempts": [60.0, 70.0, 80.0],
            "status": "completed",
            "category": "professional"
        },
        {
            "id": 2,
            "name": "Марія",
            "attempts": [40.0, 50.0, 60.0],
            "status": "completed",
            "category": "senior"
        },
        {
            "id": 3,
            "name": "Андрій",
            "attempts": [30.0, 40.0],
            "status": "disqualified",
            "category": "junior"
        }
    ]


# Тест 1. Перевірка агрегації
def test_calculate_average():
    assert calculate_average([60.0, 70.0, 80.0]) == 70.0
    assert calculate_average([]) == 0.0


# Тест 2. Перевірка бонусів
def test_calculate_bonus():
    assert calculate_bonus("junior") == 5.0
    assert calculate_bonus("senior") == 10.0
    assert calculate_bonus("professional") == 15.0
    assert calculate_bonus("unknown") == 0.0


# Тест 3. Перевірка штрафів
def test_calculate_penalty():
    athlete = {
        "id": 1,
        "name": "Тест",
        "attempts": [10.0, 20.0, 30.0],
        "status": "completed",
        "category": "junior"
    }

    assert calculate_penalty(athlete) == 0.0

    athlete_many_attempts = {
        **athlete,
        "attempts": [10.0, 20.0, 30.0, 40.0, 50.0, 60.0]
    }

    assert calculate_penalty(athlete_many_attempts) == 5.0


# Тест 4. Перевірка кваліфікації
def test_check_qualification():
    assert check_qualification(75.0, "professional") is True
    assert check_qualification(50.0, "professional") is False
    assert check_qualification(40.0, "junior") is True


# Тест 5. Перевірка референтної прозорості
def test_referential_transparency(sample_athletes):
    processor = create_default_processor()

    result1 = processor(sample_athletes)
    result2 = processor(sample_athletes)

    assert result1 == result2
```



```python
Click to add a breakpoint

# Тест 6. Перевірка відсутності мутацій
def test_no_mutation(sample_athletes):
    processor = create_default_processor()

    original = deepcopy(sample_athletes)

    processor(sample_athletes)

    assert sample_athletes == original


# Тест 7. Перевірка фільтрації за статусом
def test_status_filter(sample_athletes):
    processor = create_default_processor()

    result = processor(sample_athletes)

    assert result["count"] == 2


# Тест 8. Перевірка Callable-політик
def test_callable_policies(sample_athletes):
    processor = make_processor(
        aggregate=lambda attempts: sum(attempts),
        apply_bonus=lambda category: 10.0,
        apply_penalty=lambda athlete: 0.0,
        qualify=lambda result, category: result >= 100
    )

    result = processor(sample_athletes)

    athletes = result["results"]

    assert athletes[0]["average_result"] == 210.0
    assert athletes[0]["bonus"] == 20.0
    assert athletes[0]["final_result"] == 230.0
    assert athletes[0]["qualified"] is True


# Тест 9. Перевірка створення нового результату
def test_new_result_does_not_modify_original(sample_athletes):
    processor = create_default_processor()

    result = processor(sample_athletes)

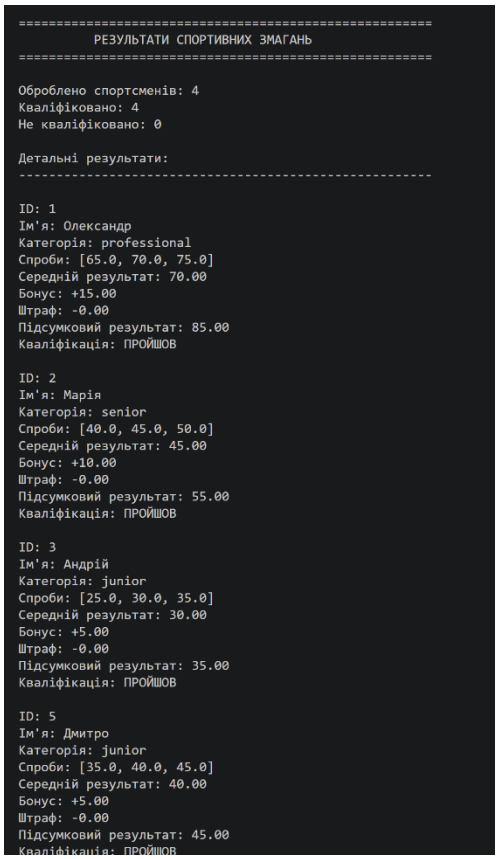
    assert "final_result" not in sample_athletes[0]
    assert "final_result" in result["results"][0]
    assert sample_athletes[0]["attempts"] is not result["results"][0]["attempts"]


# Тест 10. Перевірка порожнього списку
def test_empty_athletes():
    processor = create_default_processor()

    result = processor([])

    assert result["count"] == 0
    assert result["qualified_count"] == 0
    assert result["not_qualified_count"] == 0
    assert result["results"] == []
```

Результат:



```
======================================================
         РЕЗУЛЬТАТИ СПОРТИВНИХ ЗМАГАНЬ
======================================================

Оброблено спортсменів: 4
Кваліфіковано: 4
Не кваліфіковано: 0

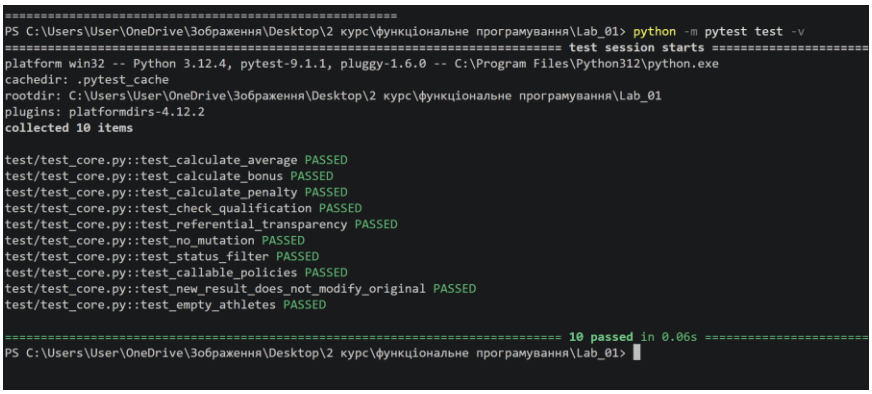
Детальні результати:
------------------------------------------------------

ID: 1
Ім'я: Олександр
Категорія: professional
Спроби: [65.0, 70.0, 75.0]
Середній результат: 70.00
Бонус: +15.00
Штраф: -0.00
Підсумковий результат: 85.00
Кваліфікація: ПРОЙШОВ

ID: 2
Ім'я: Марія
Категорія: senior
Спроби: [40.0, 45.0, 50.0]
Середній результат: 45.00
Бонус: +10.00
Штраф: -0.00
Підсумковий результат: 55.00
Кваліфікація: ПРОЙШОВ

ID: 3
Ім'я: Андрій
Категорія: junior
Спроби: [25.0, 30.0, 35.0]
Середній результат: 30.00
Бонус: +5.00
Штраф: -0.00
Підсумковий результат: 35.00
Кваліфікація: ПРОЙШОВ

ID: 5
Ім'я: Дмитро
Категорія: junior
Спроби: [35.0, 40.0, 45.0]
Середній результат: 40.00
Бонус: +5.00
Штраф: -0.00
Підсумковий результат: 45.00
Кваліфікація: ПРОЙШОВ
```



```
==================================================
PS C:\Users\User\OneDrive\Зображення\Desktop\2 курс\функціональне програмування\Lab_01> python -m pytest test -v
============================= test session starts ==============================
platform win32 -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Program Files\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\User\OneDrive\Зображення\Desktop\2 курс\функціональне програмування\Lab_01
plugins: platformdirs-4.12.2
collected 10 items

test/test_core.py::test_calculate_average PASSED
test/test_core.py::test_calculate_bonus PASSED
test/test_core.py::test_calculate_penalty PASSED
test/test_core.py::test_check_qualification PASSED
test/test_core.py::test_referential_transparency PASSED
test/test_core.py::test_no_mutation PASSED
test/test_core.py::test_status_filter PASSED
test/test_core.py::test_callable_policies PASSED
test/test_core.py::test_new_result_does_not_modify_original PASSED
test/test_core.py::test_empty_athletes PASSED

============================== 10 passed in 0.06s ==============================
PS C:\Users\User\OneDrive\Зображення\Desktop\2 курс\функціональне програмування\Lab_01>
```

Висновок. У ході лабораторної роботи було реалізовано програму для обробки спортивних результатів відповідно до 15-го варіанта. Програма здійснює агрегацію результатів спроб спортсменів, нараховує бонуси та штрафи, обчислює підсумковий результат і визначає кваліфікацію учасників залежно від їхньої категорії.

Під час розробки було застосовано принципи функціонального програмування: чисті функції, референтну прозорість, відсутність мутації вхідних даних та ізоляцію побічних ефектів. Для параметризації правил обробки використано typing.Callable, а для створення налаштованого обробника — функцію вищого порядку make_processor.

Також було реалізовано окрему оболонку програми для виведення результатів та створено автоматизовані тести за допомогою pytest. Таким чином, реалізація демонструє можливість гнучкої обробки спортивних результатів із дотриманням основних принципів функціонального програмування.